Improving Long-Context Preference Following in LLMs via a Trained Preference-Detection Classifier and Enhanced Retrieval

Krishna Jhanwar - 12341250

***Abstract*—Large language models (LLMs) used as assistants are expected to remember what a user has told them, yet PrefEval (ICLR 2025) shows that their preference following collapses as unrelated conversation accumulates. This project addresses two weaknesses that the paper identifies: an outdated retriever and the absence of any persistent preference state. First, I rebuilt PrefEval's classification benchmark to run fully locally and reproduced its five baselines with two of the paper's models (Llama-3-8B, Mistral-7B-v0.2; 64,535 questions). Second, I trained a small classifier that detects preference-stating user turns (DistilBERT, F1 0.969 on unseen topics, 1.4 ms per message). Third, the classifier feeds a structured *preference memory* that is restated before each question, and a modern encoder (EmbeddingGemma-2) replaces the paper's 2021 SimCSE retriever. On the same questions, prompts and metric as the baselines, the memory beats every baseline at every context length from 1k tokens up, for both models and both preference forms. It reaches 88.6% at 23k tokens with Mistral, versus 53.7% for the paper's best method (RAG). Ablations show that the new encoder alone fixes the retrieval bottleneck for explicit preferences, that adding retrieved raw messages on top of memory does not help, and that retrieving *over* the memory keeps accuracy stable when the memory holds many preferences.**

***Index Terms*—Personalization, preference following, long context, retrieval-augmented generation, conversational memory, text classification.**

I. Introduction

AN assistant that is told *"I avoid gluten because of a severe intolerance"* and later recommends a bakery has failed the user. PrefEval [1] measures exactly this. A preference is stated at the start of a conversation, unrelated multi-session chat follows, and a later question can only be answered well if the preference is remembered. Even strong models degrade quickly, and the paper's best mitigation, retrieval-augmented generation (RAG) with a SimCSE [2] encoder, also loses accuracy as the conversation grows.

Project criterion. PrefEval's code is public, so this project follows the criterion *"code available: address its limitations and propose a solution"*. It first reproduces the baselines locally, then proposes and evaluates an improvement.

Team and contributions. This is an individual project. Krishna Jhanwar (12341250, `krishnaj@iitbhilai.ac.in`) is responsible for all parts: problem definition and proposal, design decisions, data audit, running all experiments, analysis and reports. Tools and major prompts used are listed in the Appendix.

Problem statement and scope. The goal is to improve preference following in long conversations for open 7–8B models running on a single consumer GPU. The scope is PrefEval's *classification* (multiple-choice) task, for explicit and implicit choice-based preferences, at the paper's context lengths of 0.2k–23k tokens. The generation task is out of scope because its judge (Claude-3-Sonnet via Bedrock) is not reproducible locally. The problem matters for ML because it isolates two open issues in deployed LLMs: using information far back in a long context [3], and maintaining user state over many sessions.

Contributions. (1) A fully local, verified reproduction of PrefEval's classification baselines (64,535 questions, two models, five methods). (2) A preference-detection classifier, trained with topic-held-out cross-validation and audited pseudo-labels, that is 51× faster and more accurate than prompting an 8B LLM. (3) A retrieval benchmark of five current encoders against the paper's retriever. (4) A classifier-driven preference memory, evaluated against all baselines on 46,000 further questions, with ablations and a multi-preference stress test.

II. Related Work

Preference following. PrefEval [1] evaluates explicit and implicit preferences over long multi-session contexts and compares zero-shot, reminder, self-critic, few-shot chain-of-thought and RAG prompting. **Long context.** Models use information at the start of long inputs poorly [3]; PrefEval places the preference exactly there. **Retrieval.** RAG adds retrieved text to the prompt. PrefEval uses SimCSE [2]; since then, retrieval-trained encoders such as BGE [4], E5 [5], mxbai-embed-large [6] and EmbeddingGemma [7] have become standard. **Memory.** Systems such as MemGPT [8] let an LLM write and read long-term memory, which costs extra LLM calls. This project instead uses a small *trained* classifier to decide what is stored, which needs one cheap forward pass per turn.

III. Proposed Methodology

The work has three phases; each builds on the previous one (Fig. 1).

A. Phase 1: local reproduction of the baselines

PrefEval's released code calls Amazon Bedrock for every model, and an earlier local attempt failed with several unrelated errors. I rebuilt the classification pipeline around the *original* prompt-building functions (vendored unchanged from the upstream repository) and replaced only the inference call

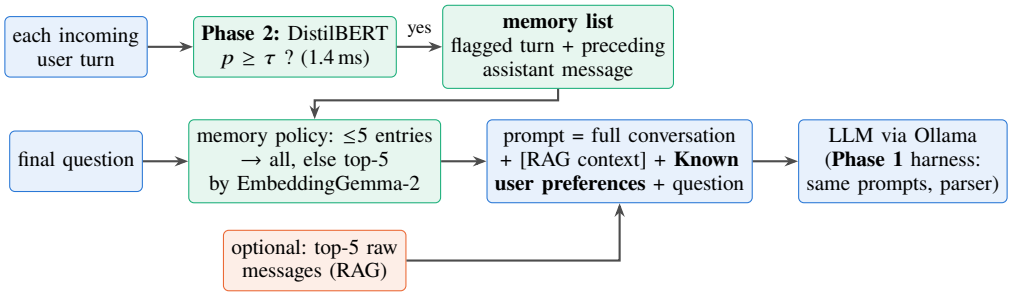


Fig. 1. Proposed pipeline. Phase 1 provides the local evaluation harness and baselines, Phase 2 the classifier, Phase 3 the memory, the memory policy and the retriever (green: new components; orange: retrieval over raw messages, evaluated as an ablation).

with Ollama in raw-prompt mode. The model therefore sees exactly the text that Bedrock received. Answers are parsed with the original strict `<choice>X</choice>` parser. Every question is written to disk as soon as it is answered, so a run resumes after any interruption. Five issues that would have silently biased the reproduction were found and fixed:

- a duplicated start-of-sequence token;
- trailing whitespace in the Llama self-critic templates, which made the model stop immediately;
- silent prompt truncation at long context;
- two defects in the released retrieval data (one truncated file, seven files that end early);
- a GPU-memory limit at 32k context, solved with flash attention and an 8-bit KV cache for Mistral, enforced by a per-model settings guard.

B. Phase 2: preference-detection classifier

Task: given one user message (text only), decide whether it states a lasting personal preference. *Data:* 12,950 messages. 3,580 labels come from PrefEval's structure (preference sentences and choice selections are positive; queries and questions negative). 9,188 are pseudo-labels from Llama-3-8B for persona-dialogue turns and 5,000 OpenAssistant messages. A manual audit of 200 pseudo-labels exposed two systematic error types; targeted refinement raised agreement with the human labels from 73.9% to 90.7%, and 182 rows carry the human label. *Leakage control:* topic-level 5-fold cross-validation (test topics are never seen in training), and the LMSYS distractor turns are never trained on. *Models:* DistilBERT [9], ModernBERT [10], and TinyLlama-1.1B [11] and Qwen3-1.7B [12] with LoRA [13]. The decision threshold is tuned on each fold's validation topics.

C. Phase 3: preference memory and modern retrieval

Step 1, retrieval benchmark (no LLM calls). Five encoders (bge-base/large-v1.5 [4], e5-base-v2 [5], mxbai-embed-large [6], EmbeddingGemma-2 [7]) are compared with the paper's stored SimCSE scores. Each one must find the preference message among all messages before the question, using each model's official query/document prompts and weights pinned to an exact version. Because a bi-encoder embeds each message independently, pre-computing embeddings gives exactly what a live system computes turn by turn; the real-time cost is reported as latency.

Step 2, memory. Each user turn is scored by the Phase 2 classifier. For a question of topic t, the fold model that was *tested* on t is used, so the classifier never saw that topic. A flagged turn is stored together with the assistant message just before it. For implicit preferences, the entry is therefore the options list plus the user's selection: Step 1 showed that the selection alone (e.g. "I'll take the second one") is meaningless without the options. At question time, all entries go into the prompt if there are at most five; otherwise the five most similar to the question are selected with EmbeddingGemma-2. They are inserted as a short "Known user preferences" block right before the question.

Methods compared. Each new method changes exactly one thing compared with a Phase 1 baseline: (i) *Memory* = the zero-shot prompt plus the memory block; (ii) *RAG-Gemma* = the paper's RAG prompt with EmbeddingGemma-2 replacing SimCSE; (iii) *Memory+RAG* = (ii) plus the memory block. With SimCSE's retrieved messages substituted, our RAG prompt is byte-identical to the Phase 1 RAG prompt, and the memory prompt with an empty memory is byte-identical to the zero-shot prompt (checked on 96 prompts). Every accuracy difference is therefore caused by the method itself.

IV. Experimental Setup

Dataset. PrefEval [1] (public, in the GitHub repository and on HuggingFace): 1,000 preference–question pairs over 20 topics in each preference form. Each question has four options, exactly one of which respects the preference. Distractor turns come from LMSYS-Chat-1M [14]. The paper's context rows 0.2k/1k/3k/10k/16k/23k tokens correspond to 0/3/8/28/48/68 inserted turn pairs. The explicit and implicit choice-based forms are used; Phase 2 additionally uses OpenAssistant `oasst1`.

Models. Llama-3-8B-Instruct [15] (8k context, so 0.2k–3k only) and Mistral-7B-Instruct-v0.2 [16] (up to 23k), both used in the paper, served as 8-bit weights by Ollama with greedy decoding (temperature 0, seed 41). Mixtral-8x7B does not fit the hardware.

Metric. Accuracy with the paper's strict answer parser, macro-averaged over the 20 topics, as in the paper. A lenient parse is logged as a diagnostic only. The classifier is evaluated with precision, recall and F1 on unseen-topic test folds.

Protocol. Every Phase 3 cell uses the same 1,000 questions, option order and prompts as its Phase 1 baseline, so all

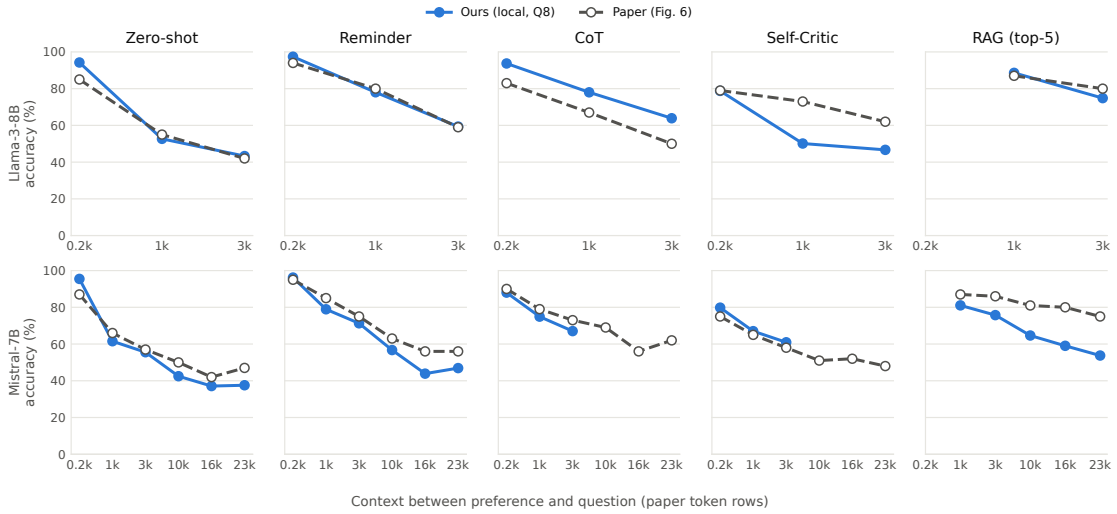


Fig. 2. Phase 1: explicit-preference accuracy, our local reproduction (solid) vs. the paper's Figure 6 (dashed).

comparisons are paired. The paper's RAG has stored scores for 949 explicit and 973 implicit questions; RAG comparisons are additionally reported on exactly those.

Resources. One workstation: NVIDIA RTX 3060 (12 GB), Intel i7-12700, 15 GB RAM. Software: Python 3.11, PyTorch, HuggingFace `transformers` and `peft`, sentence-transformers, scikit-learn, Ollama. In total, about 120 GPU-hours: Phase 1 71 h, Phase 3 44 h, plus classifier training and pseudo-labelling. All long runs ran in `tmux` and resume per question.

V. Results and Analysis

A. Phase 1: reproduction

Of the 29 cells with a published number (excluding the trivial 0.2k row), 12 are within 5 points of the paper and 20 within 10 (median absolute difference 6.0 points; Fig. 2, Table I). The paper's qualitative findings reproduce:

- accuracy collapses with context length (Llama zero-shot: 94.2% at 0.2k → 43.3% at 3k);
- Reminder and RAG are the strongest baselines;
- self-critique often overturns correct answers;
- implicit preferences are harder.

The largest gap, Mistral RAG at long context (53.7% vs. 75%), coincides with the 8-bit KV cache needed to fit 32k tokens in 12 GB. At long context the model also falls back on answer position: Mistral zero-shot is right 42% of the time when the answer is A, but only 23% when it is D.

B. Phase 2: classifier

All four classifiers reach F1 0.969–0.976 on unseen topics, and their differences are small (Table II). On the same 2,000 messages, a zero-shot prompted Llama-3-8B reaches only F1 0.879 and wrongly flags 16.5% of non-preference messages, while DistilBERT is 51× faster (1.4 ms vs. 72 ms per message). On the LMSYS turns that Phase 3 replays, the models raise false alarms on only 0.1–1.5% of messages.

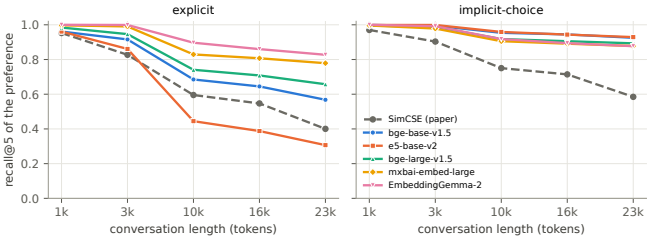


Fig. 3. Recall@5 of the preference message versus conversation length.

DistilBERT was selected for Phase 3 as the fastest model with a low false-alarm rate.

C. Phase 3, Step 1: retrieval

EmbeddingGemma-2 finds the explicit preference in its top 5 for 82.7% of questions at 23k tokens, versus 40.0% for SimCSE (implicit: 87.7% vs. 58.5%; Table III, Fig. 3). SimCSE almost never ranks the preference first (recall@1 8.5% at 1k, vs. 82.4%). Newer is not automatically better: e5-base-v2 falls below SimCSE on long explicit conversations (30.7% at 23k). EmbeddingGemma-2 was chosen (15.6 ms per message). On implicit preferences, no encoder ranks the user's selection first (recall@1 5–28%), which motivated storing it together with the options message. Verifying the benchmark also revealed an upstream data quirk: in 13 of 20 topics, the file that maps SimCSE scores back to message text is shifted by one exchange. Recall and rank are unaffected, because they depend only on the scores.

D. Phase 3, Step 2: memory quality

The classifier places the true preference in memory for 98.6% of explicit and 99.9% of implicit questions. The 14 explicit misses are mostly phrased as dislikes, e.g. "I'm uncomfortable with invisible fences". False entries appear only at 16k (0.22 per question) and 23k (0.65): two LMSYS messages

TABLE I
PHASE 1 REPRODUCTION, EXPLICIT PREFERENCE: OURS / PAPER (DIFFERENCE), ACCURACY %.

Llama-3-8B-Instruct

Method	0.2k (0)	1k (3)	3k (8)
Zero-shot	94.2 / 85 (+9.2)	52.7 / 55 (−2.3)	43.3 / 42 (+1.3)
Reminder	97.4 / 94 (+3.4)	78.0 / 80 (−2.0)	59.4 / 59 (+0.4)
CoT	93.7 / 83 (+10.7)	78.0 / 67 (+11.0)	63.9 / 50 (+13.9)
Self-Critic	78.8 / 79 (−0.2)	50.1 / 73 (−22.9)	46.7 / 62 (−15.3)
RAG (top-5)	–	88.6 / 87 (+1.6)	74.8 / 80 (−5.2)

Mistral-7B-Instruct-v0.2

Method	0.2k (0)	1k (3)	3k (8)	10k (28)	16k (48)	23k (68)
Zero-shot	95.5 / 87 (+8.5)	61.5 / 66 (−4.5)	55.5 / 57 (−1.5)	42.5 / 50 (−7.5)	37.1 / 42 (−4.9)	37.6 / 47 (−9.4)
Reminder	96.2 / 95 (+1.2)	78.9 / 85 (−6.1)	71.3 / 75 (−3.7)	56.7 / 63 (−6.3)	43.9 / 56 (−12.1)	46.9 / 56 (−9.1)
CoT	88.0 / 90 (−2.0)	74.9 / 79 (−4.1)	67.0 / 73 (−6.0)	–	–	–
Self-Critic	79.8 / 75 (+4.8)	67.0 / 65 (+2.0)	61.0 / 58 (+3.0)	–	–	–
RAG (top-5)	–	81.0 / 87 (−6.0)	75.7 / 86 (−10.3)	64.6 / 81 (−16.4)	59.0 / 80 (−21.0)	53.7 / 75 (−21.3)

TABLE II
PHASE 2: PREFERENCE-DETECTION CLASSIFIERS, POOLED OVER UNSEEN-TOPIC TEST FOLDS. OOD FLAGS = FALSE ALARMS ON LMSYS DISTRACTOR TURNS.

Model	F1	Prec.	Rec.	F1 std	AUC	ECE	OOD flags	ms/msg	Trainable	Train min
DistilBERT	0.969	0.983	0.955	0.007	0.994	0.008	0.4%	1.4	67M	14
ModernBERT	0.972	0.988	0.956	0.004	0.995	0.009	0.2%	6.5	150M	32
TinyLlama-1.1B (LoRA)	0.974	0.988	0.961	0.006	0.997	0.010	0.1%	17.1	13M	91
Qwen3-1.7B (LoRA)	0.976	0.988	0.965	0.006	0.997	0.009	1.5%	25.1	17M	120

TABLE III
PHASE 3, STEP 1: RETRIEVAL OF THE PREFERENCE MESSAGE (RECALL@5 UNLESS NOTED), ON THE QUESTIONS WITH STORED SIMCSE SCORES.

		Explicit					Implicit-choice				
Retriever	Params	R@5 1k	3k	10k	23k	R@1 23k	R@5 1k	3k	10k	23k	ms/msg
SimCSE (paper)	355M	0.95	0.83	0.60	0.40	0.05	0.97	0.90	0.75	0.58	–
bge-base-v1.5	110M	0.96	0.92	0.68	0.57	0.29	1.00	0.99	0.95	0.92	3.5
e5-base-v2	110M	0.96	0.86	0.44	0.31	0.12	1.00	1.00	0.96	0.93	3.6
bge-large-v1.5	335M	0.98	0.95	0.74	0.66	0.43	1.00	0.99	0.92	0.89	9.8
mxbai-embed-large	335M	1.00	0.99	0.83	0.78	0.53	1.00	0.98	0.91	0.88	9.8
EmbeddingGemma-2	744M	1.00	1.00	0.90	0.83	0.53	1.00	0.99	0.92	0.88	15.6

TABLE IV
LLAMA-3-8B, ACCURACY % (MACRO OVER 20 TOPICS). BOLD: BEST PER COLUMN.

	Explicit			Implicit-choice		
Method	0.2k	1k	3k	0.2k	1k	3k
Zero-shot	94.2	52.7	43.3	77.0	53.3	46.0
Reminder	97.4	78.0	59.4	**91.0**	77.0	60.3
CoT	93.7	78.0	63.9	84.4	70.3	61.7
Self-Critic	78.8	50.1	46.7	66.1	50.1	38.7
RAG, SimCSE (paper)	–	88.6	74.8	–	82.8	73.7
Memory (ours)	**97.7**	**96.0**	**96.5**	90.0	**86.3**	**83.5**
RAG, EmbeddingGemma-2 (ours)	–	94.7	93.9	–	83.6	79.1
Memory + RAG (ours)	–	93.4	92.9	–	84.2	75.4

that read like stated opinions. Memories therefore hold 1–3 entries.

E. Phase 3: main comparison with the baselines

Tables IV–V and Fig. 4 give the end-to-end results.

- **Memory beats every baseline at every length from 1k tokens up**, in all 11 such cells across both models and both forms. It is the best of all methods in 10 of them; at Mistral 10k, Memory+RAG is 0.4 points higher.
- **Most of the long-context collapse is removed.** With Mistral at 23k tokens, memory reaches 88.6%, versus 53.7% for RAG, 46.9% for Reminder and 37.6% for zero-shot. From 0.2k to 23k, zero-shot and Reminder lose 49–58 points; memory loses 8.5. With Llama at 3k: 96.5% vs. 74.8% (explicit) and 83.5% vs. 73.7% (implicit).
- **These results are also above the paper's own published numbers** for the same models and lengths (e.g. paper RAG: 80% for Llama at 3k, 75% for Mistral at 23k).
- **With no distractors (0.2k) there is nothing to forget**: memory only ties with Reminder on explicit, and on implicit it is lower (Mistral 81.9% vs. 88.4%).

VI. ABLATION STUDY

A. The retriever alone

With the prompt fixed and only the encoder swapped, on identical questions (Table VI), RAG improves strongly on

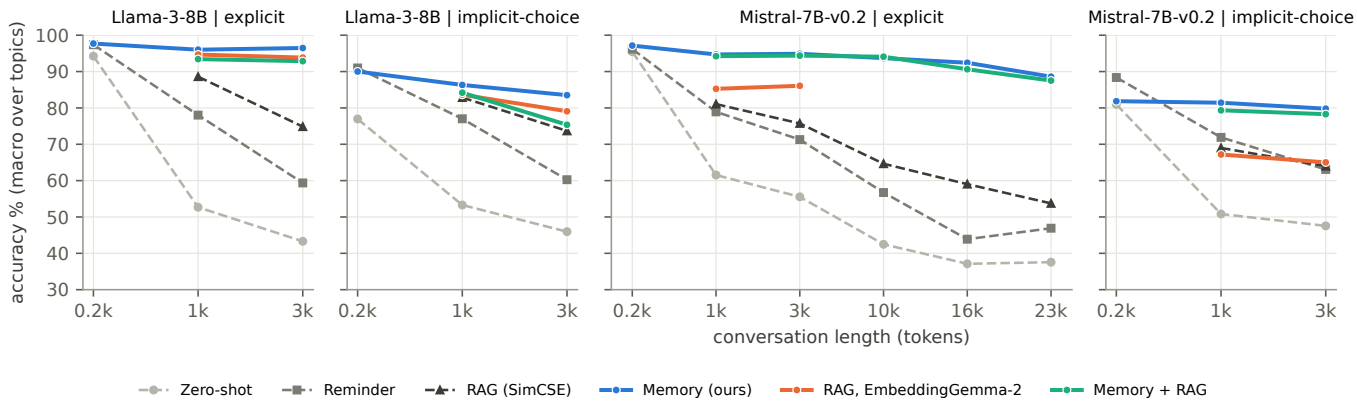


Fig. 4. Accuracy versus conversation length. Dashed gray: Phase 1 baselines; solid: proposed methods. The memory (blue) stays nearly flat.

TABLE V
MISTRAL-7B-V0.2, ACCURACY %. COT AND SELF-CRITIC WERE NOT RUN BEYOND 3K (AS IN THE PAPER).

Method	Explicit 0.2k	1k	3k	10k	16k	23k
Zero-shot	95.5	61.5	55.5	42.5	37.1	37.6
Reminder	96.2	78.9	71.3	56.7	43.9	46.9
CoT	88.0	74.9	67.0	–	–	–
Self-Critic	79.8	67.0	61.0	–	–	–
RAG, SimCSE (paper)	–	81.0	75.7	64.6	59.0	53.7
Memory (ours)	**97.1**	**94.7**	**94.9**	93.7	**92.4**	**88.6**
RAG, EmbeddingGemma-2 (ours)	–	85.3	86.1	–	–	–
Memory + RAG (ours)	–	94.2	94.4	**94.1**	90.6	87.5

Method	Implicit-choice 0.2k	1k	3k
Zero-shot	81.0	50.8	47.5
Reminder	**88.4**	71.9	63.1
CoT	74.0	60.2	59.5
Self-Critic	67.2	51.8	50.9
RAG, SimCSE (paper)	–	69.0	64.0
Memory (ours)	81.9	**81.4**	**79.8**
RAG, EmbeddingGemma-2 (ours)	–	67.2	65.0
Memory + RAG (ours)	–	79.3	78.3

explicit preferences (Llama at 3k: 74.8 → 93.9). On implicit preferences it improves little or not at all, even though retrieval itself improved: retrieved fragments of a choice dialogue are hard for the model to use. The memory entry, which restores the full options-plus-selection context, is what fixes the implicit case.

B. Adding RAG on top of memory

Memory+RAG is never clearly better than memory alone. A per-question comparison (Table VII) shows why:

- With Llama, most losses are *format* failures. With two long context blocks, the model writes "Based on your previous…" instead of `<choice>X</choice>`, and the paper's 5-token answer budget cuts it off (65 of 99 lost implicit questions at 3k).
- The preference had been retrieved in 98 of those 99 cases, so retrieval itself did not fail.
- Mistral, whose template pre-fills `<choice>`, has no format failures. There, the two methods give the same answer on 90–98% of questions, and the remaining retrieved distractors occasionally pull the model to another option.

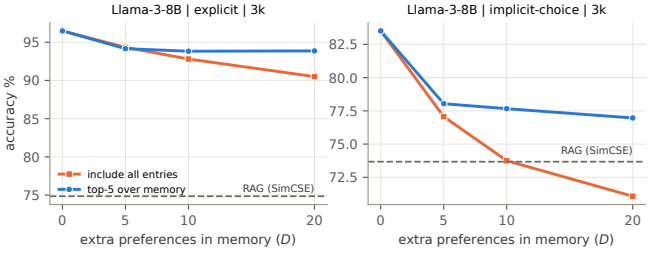


Fig. 5. Stress test: "include all" degrades as the memory grows; top-5 retrieval over the memory levels off.

With ≈99% classifier recall there is little for retrieval to recover, so retrieving raw messages mainly adds noise.

C. Memory policy under many preferences (stress test)

PrefEval conversations contain one preference each, so the "top-5 over memory" branch never triggers. To test it, $D \in \{5, 10, 20\}$ preferences from *other* topics were added to each memory, as if stated in earlier sessions. Same-topic preferences were excluded because they can contradict the true one. Entries were shown in a seeded random order without turn labels, so position gives no hint. Llama-3-8B, 3k tokens. Injecting every entry degrades steadily (Table VIII, Fig. 5): at D=20 accuracy is 90.5% (explicit) and 71.1% (implicit), even though the true preference is in the prompt. Retrieving the top 5 over the memory keeps the true preference for 97.4%/98.5% of questions and holds 93.9%/77.0%, still above the paper's RAG at the same length.

D. Classifier choice

Phase 2 compared four detectors and all 15 ensembles: the best ensemble adds only +0.001 F1, because the models' errors are highly correlated. The smallest model (DistilBERT) was therefore sufficient, and it is 51× faster than LLM-based detection.

VII. DISCUSSION

- **Why memory works.** The failure PrefEval measures is not that the model cannot apply a preference; at 0.2k

TABLE VI
RAG WITH SIMCSE VS. EMBEDDINGGEMMA-2 (SAME PROMPT, SAME QUESTIONS), AND MEMORY+RAG.

Model	Form	Length	SimCSE RAG	EmbeddingGemma-2 RAG	Δ	Memory + RAG
Llama-3-8B	explicit	1k	88.6	94.6	+6.0	93.2
Llama-3-8B	explicit	3k	74.8	93.9	+19.1	92.7
Llama-3-8B	implicit-choice	1k	82.8	84.1	+1.2	84.5
Llama-3-8B	implicit-choice	3k	73.7	79.6	+6.0	75.8
Mistral-7B-v0.2	explicit	1k	81.0	85.5	+4.5	94.2
Mistral-7B-v0.2	explicit	3k	75.7	86.3	+10.6	94.5
Mistral-7B-v0.2	implicit-choice	1k	69.0	67.6	-1.4	79.3
Mistral-7B-v0.2	implicit-choice	3k	64.0	65.6	+1.6	78.2

TABLE VII
QUESTIONS CORRECT WITH MEMORY BUT WRONG WITH MEMORY+RAG ("LOST"), SPLIT INTO FORMAT FAILURES AND WRONG OPTIONS; "PREF. RETRIEVED": THE PREFERENCE WAS IN THE RETRIEVED TOP 5; "GAINED" IS THE REVERSE.

Model	Form	Length	Lost	of which format	wrong option	pref. retrieved	Gained	Same answer %
Llama-3-8B	explicit	3k	49	40	9	49	11	93.8
Llama-3-8B	implicit-choice	3k	99	65	34	98	16	85.2
Mistral-7B-v0.2	explicit	3k	14	0	14	14	7	97.5
Mistral-7B-v0.2	explicit	23k	28	0	28	22	16	95.1
Mistral-7B-v0.2	implicit-choice	3k	46	1	45	46	26	90.0

TABLE VIII
STRESS TEST: ACCURACY % WITH D EXTRA PREFERENCES IN MEMORY. "PREF. KEPT": SHARE OF QUESTIONS WHERE TOP-5 RETRIEVAL KEEPS THE TRUE PREFERENCE.

		Include all		Top-5 retrieval over memory	
Form	D	entries	accuracy	pref. kept (%)	accuracy
explicit	0	1	96.5	100.0	96.5
	5	6	**94.3**	100.0	94.2
	10	11	92.8	99.1	**93.8**
	20	21	90.5	97.4	**93.9**
implicit-choice	0	1	83.5	100.0	83.5
	5	6	77.1	100.0	**78.0**
	10	11	73.7	99.9	**77.7**
	20	21	71.1	98.5	**77.0**

it applies it about 95% of the time. The failure is that the preference gets lost in the conversation. Restating it next to the question removes the distance between the preference and the question that causes the long-context failure.

- **What the classifier contributes.** Deciding what to store is cheap (1.4 ms per turn) and accurate (98.6–99.9% of preferences caught, almost no false entries). The 14 questions whose preference was missed are answered correctly only 36–50% of the time at long context, so the classifier is the part of the pipeline whose errors matter most.
- **When retrieval is useful.** Retrieval over raw messages is the paper's mechanism, and the new encoder makes it much better. Once a memory exists, retrieval is most useful *over the memory*, as the proposal anticipated. This keeps the prompt short when users have many preferences.
- **Cost.** At question time the memory adds about 61 prompt tokens (3,282 → 3,343). Per turn, it costs one classifier pass and one embedding, far cheaper than LLM-based memory writing.

VIII. LIMITATIONS

- **One benchmark with one preference per conversation.** Many-preference behaviour is approximated by a synthetic stress test (one model, one length, other-topic preferences only).
- **The classifier is in-distribution.** It was trained on PrefEval-style data from other topics. It catches only 16–26% of casual preferences in real chat (OpenAssistant), so memory recall in the wild would be lower.
- **The strict 5-token metric** penalizes correct but badly formatted answers. It was kept for comparability with the paper.
- **Single deterministic run per cell** (greedy decoding). Differences of 1–2 points between the proposed variants are not claimed as significant; the gaps to the baselines (10–35 points) are far larger.
- **Local setup.** Quantized 7–8B models instead of the paper's Bedrock models. The generation task, the implicit-persona form and Mistral RAG-Gemma at 10k–23k were not run.

IX. CONCLUSION AND FUTURE WORK

A cheap trained classifier that writes preferences into a memory, restated before each question, solves most of PrefEval's long-context preference-following problem for open 7–8B models: 88.6% at 23k tokens versus 53.7% for the paper's best method, at almost no extra cost. A modern encoder fixes the paper's retrieval bottleneck, but once memory exists, retrieval is best applied over the memory rather than over raw messages. Future work: evaluate the generation task with a local or API judge; improve real-chat recall of the classifier with more diverse training data; handle preference updates and conflicts (newer statements overriding older ones); and test on genuinely multi-preference, multi-session conversations.

References

[1] S. Zhao, M. Hong, Y. Liu, D. Hazarika, and K. Lin, "Do LLMs recognize your preferences? evaluating personalized preference following in LLMs," in *International Conference on Learning Representations (ICLR)*, 2025.

[2] T. Gao, X. Yao, and D. Chen, "SimCSE: Simple contrastive learning of sentence embeddings," in *Proc. EMNLP*, 2021.

[3] N. F. Liu, K. Lin, J. Hewitt *et al.*, "Lost in the middle: How language models use long contexts," in *Transactions of the Association for Computational Linguistics*, vol. 12, 2024.

[4] S. Xiao, Z. Liu, P. Zhang, N. Muennighoff, D. Lian, and J.-Y. Nie, "C-Pack: Packed resources for general Chinese embeddings," in *Proc. SIGIR*, 2024.

[5] L. Wang, N. Yang, X. Huang *et al.*, "Text embeddings by weakly-supervised contrastive pre-training," *arXiv preprint arXiv:2212.03533*, 2022.

[6] S. Lee, A. Shakir, and D. Koenig, "Open source strikes bread – new fluffy embedding model," Mixedbread blog, https://www.mixedbread.ai/blog/mxbai-embed-large-v1, 2024.

[7] Google, "EmbeddingGemma-2 model card," Hugging Face, https://huggingface.co/google/embeddinggemma-2, 2026.

[8] C. Packer, S. Wooders, K. Lin *et al.*, "MemGPT: Towards LLMs as operating systems," *arXiv preprint arXiv:2310.08560*, 2023.

[9] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019.

[10] B. Warner, A. Chaffin, B. Clavié *et al.*, "Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference," *arXiv preprint arXiv:2412.13663*, 2024.

[11] P. Zhang, G. Zeng, T. Wang, and W. Lu, "TinyLlama: An open-source small language model," *arXiv preprint arXiv:2401.02385*, 2024.

[12] Qwen Team, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.

[13] E. J. Hu, Y. Shen, P. Wallis *et al.*, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2022.

[14] L. Zheng, W.-L. Chiang, Y. Sheng *et al.*, "LMSYS-Chat-1M: A large-scale real-world LLM conversation dataset," in *International Conference on Learning Representations (ICLR)*, 2024.

[15] Llama Team, AI @ Meta, "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[16] A. Q. Jiang, A. Sablayrolles, A. Mensch *et al.*, "Mistral 7B," *arXiv preprint arXiv:2310.06825*, 2023.

Appendix A
Platforms and Tools

Experiments ran on a local RTX 3060 workstation, with Ollama (local LLM serving), HuggingFace (models and datasets) and GitHub (version control). Claude Code (Anthropic) was used as an AI assistant for implementation and writing; the major prompts are listed in Appendix B. Code and results: https://github.com/krishnaj01/ML_Project_PrefEval.

Key design decisions. Local-only evaluation with Ollama; adding Mistral-7B and the long-context regime (Mixtral ruled out after a feasibility check); four classifiers with ensemble analysis and OpenAssistant data; the strict persona-label filter after a hand-labelled 200-row audit; adding mxbai-embed-large to the encoder comparison; single-message retrieval units; memory entries stored with their context; the "Known user preferences" format; comparison on the full Phase 1 grid; and a multi-preference stress test instead of an oracle-memory bound.

Appendix B
Major Prompts Given to the LLM

Main instructions, lightly edited for spelling (routine follow-ups and status checks omitted).

1) *Setup.* "This proposal is the SoP for my ML project based on the existing PrefEval paper. Use Ollama for running the model if possible, and start from scratch with the environment. I have to run this with a local model, with proper logging, result storing and final outputs to compare with the baseline for Phase 1. Tell me which tasks are possible without Amazon Bedrock. Give time estimates and tmux commands for long-running tasks, save progressively with a resume option, and tell me when to push to GitHub."
2) *Second model.* "Can we run the same full Phase 1 tiers for Mistral-7B-Instruct-v0.2? Give a proper feasibility analysis, and tell me whether Mixtral-8x7B is possible given the system constraints."
3) *Phase 2.* "Let's move on to Phase 2, where I have to train a small classifier. From the four base-model options, give proper reasoning and trade-offs so that I can choose the best for our use case, and do real analysis of training and testing: loss curves, accuracy, precision/recall, F1, confusion matrix."
4) *Audit.* "When exactly should I put 1 and when 0 in the audit sample, and what if I am unsure? Why is this audit required?"
5) *Phase 3 plan.* "Let's plan first: start with the retrieval part, and also add mxbai-embed-large-v1 for comparison, so five models in total; then we discuss prompt making and memory. Will embedding the text before the conversation create issues, since it is not real-time? How did the paper do it with SimCSE?"
6) *Memory design.* "Store the context with each preference. The 'Known user preferences' format works. How many preferences should go in, since it depends on which preferences occur, where and when: include all, or do RAG over the memory? Should the evaluation be the full Phase 1 comparison, or what does the proposal suggest?"
7) *Final scope.* "Keep the oracle memory out, but explain exactly what it means first. Include the multi-preference stress test."